

Data Model

Source: docs/data-model.md

Data Model

Status: internal technical documentation Last updated: 2026-05-23 Extended reference: ARCHITECTURE_DEEP_DIVE.md

This document reflects the active schema used by the current code. Older materials may refer to legacy table names; the active application uses the tables below.

Active Tables

app_users

Authentication and role records.

Column	Type	Notes
id	uuid	Primary key
username	text	Login identifier
email	text	Optional/contact email
password_hash	text	bcrypt hash
role	text	Usually user or admin
created_at	timestamp	Creation time

user_documents

Metadata for user-uploaded documents.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	Owner, references app_users.id
filename	text	Original file name
file_type	text	File extension/type
tag	text	ESG tag used for retrieval filters
status	text	Processing state
created_at	timestamp	Upload time

user_document_chunks

Vectorized chunks from user documents.

Column	Type	Notes
id	uuid	Primary key
document_id	uuid	References user_documents.id
user_id	uuid	Owner for filtering
chunk_text	text	Parsed chunk content
embedding	vector	OpenAI text-embedding-3-small output
chunk_index	integer	Position in document
metadata	json/jsonb	Source details when available
tag	text	ESG filter tag when available
created_at	timestamp	Creation time

knowledge_documents

Metadata for administrator-managed ESG knowledge-base documents.

Column	Type	Notes
id	uuid	Primary key
filename	text	Source file name
document_type	text	Knowledge document category
tag	text	Optional ESG/general tag
version	text	Version label
uploaded_by	uuid	Admin user id
created_at	timestamp	Upload time

knowledge_chunks

Vectorized chunks from the shared ESG knowledge base.

Column	Type	Notes
id	uuid	Primary key
document_id	uuid	References knowledge_documents.id
chunk_text	text	Chunk content
embedding	vector	OpenAI text-embedding-3-small output
chunk_index	integer	Position in document
metadata	json/jsonb	Version/source metadata
tag	text	Optional ESG/general tag
created_at	timestamp	Creation time

reports

Stored report history.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	Owner, references app_users.id
input_text	text	Generation request summary
response_text	text	Raw JSON string returned by the LLM
report_type	text	ESG, Environmental, Social or Governance
used_chunks	text	JSON-encoded list of RAG chunks
created_at	timestamp	Creation time

chat_sessions

Chat session containers.

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	Owner, references app_users.id
title	text	Short title derived from the first query
created_at	timestamp	Creation time

chat_messages

Messages inside a chat session.

Column	Type	Notes
id	uuid	Primary key
session_id	uuid	References chat_sessions.id
role	text	user or assistant
content	text	Message content
created_at	timestamp	Creation time

Relationships

```
[text]
app_users
|--< user_documents
|   |--< user_document_chunks
|
|--< reports
|
|--< chat_sessions
|   |--< chat_messages

knowledge_documents
|--< knowledge_chunks
```

Vector Search

The application retrieves RAG context through Supabase RPC `match_chunks2`. `backend/RAG/rag_retriever.py` sends:

- `query_embedding`
- `match_threshold`
- `match_count`
- `filter_tag`
- `query_user_id`

The RPC returns chunk rows that are formatted by the Python retriever as:

```
[text]
--- DOKUMENT: <source> ---
<chunk_text>
```

Report generation then separates company data from legal/knowledge-base context before prompting the model. Until RPC results expose structured source metadata, this split is header-based.